



PROBLEM SESSION 4: Memory class and API in C programming.

Member	Student ID
1. Kulchaya Paipinij	67070503406
2. Chayanit Kuntanarumitkul	67070503408
3. Siripitch Chaiyabutra	67070503440

1. Static Storage Class

Step 1: Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way

Code:

```
UW PICO 5.09      File: static.c

#include <stdio.h>
#include <stdlib.h>

void main()
{
    auto int x = 3;

    while (x > 0)
    {
        static int y = 5;

        y++;

        printf("The value of y is %d\n", y);
        printf("The address of y is %p\n\n", &y);

        x--;
    }
}
```

^G Get Help ^O Write Out ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where i ^V Next Pg ^U UnCut T ^T To Spel

Output:

```
1 warning generated.
[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x10ca63000

The value of y is 7
The address of y is 0x10ca63000

The value of y is 8
The address of y is 0x10ca63000

[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x10b3e6000

The value of y is 7
The address of y is 0x10b3e6000

The value of y is 8
The address of y is 0x10b3e6000

[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x10ba49000

The value of y is 7
The address of y is 0x10ba49000

The value of y is 8
The address of y is 0x10ba49000

[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % nano static.c
```

Explanation:

The variable **y** is declared as **static**, so it is initialized only once with the value 5 and retains its value between loop iterations. Therefore, after **y++**, the values become **6, 7, and 8**.

The address of **y** remains the same within each execution because the same memory location is used. However, the address changes between executions because the operating system may assign different virtual memory addresses due to ASLR.

Step 2: Modify the code by delete the “static” word, compile, and re-run the program.

Modified Code:

UW PICO 5.09File: static.cModified

```
#include <stdio.h>
#include <stdlib.h>

void main()
{
    auto int x = 3;

    while (x > 0)
    {
        int y = 5;

        y++;

        printf("The value of y is %d\n", y);
        printf("The address of y is %p\n\n", &y);

        x--;
    }
}
```

^G Get Hel^O WriteOu^R Read Fi^Y Prev Pg^K Cut Tex^C Cur Pos
^X Exit ^J Justify^W Where i^V Next Pg^U UnCut T^T To Spel

Step 3: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Output:

```
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x7ff7b144d6e8

The value of y is 6
The address of y is 0x7ff7b144d6e8

The value of y is 6
The address of y is 0x7ff7b144d6e8

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x7ff7b9aca6e8

The value of y is 6
The address of y is 0x7ff7b9aca6e8

The value of y is 6
The address of y is 0x7ff7b9aca6e8

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static
The value of y is 6
The address of y is 0x7ff7bba106e8

The value of y is 6
The address of y is 0x7ff7bba106e8

The value of y is 6
The address of y is 0x7ff7bba106e8
```

Explanation:

When the **static** keyword is removed, **y** becomes a local automatic variable and is initialized to 5 each time the loop iteration begins. Therefore, after **y++**, the value of **y** is always 6.

The address remains the same in each iteration in our output because the compiler can reuse the same stack memory location. The main difference is that the static variable retains its value between iterations (6, 7, 8), while the non-static variable is reset each iteration (6, 6, 6).

Step 4: Redo step 1-3, but this time compile the program using:

“gcc -no-pie -o <your executable object name> <your c code name>”

Are they different? Why or why not

Command used :

```
gcc -no-pie -o static-nopie static.c
```

Output:

```
[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static-nopie
The value of y is 6
The address of y is 0x7ff7be13e6d8

The value of y is 6
The address of y is 0x7ff7be13e6d8

The value of y is 6
The address of y is 0x7ff7be13e6d8

[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static-nopie
The value of y is 6
The address of y is 0x7ff7b10346d8

The value of y is 6
The address of y is 0x7ff7b10346d8

The value of y is 6
The address of y is 0x7ff7b10346d8

[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./static-nopie
The value of y is 6
The address of y is 0x7ff7b207d6d8

The value of y is 6
The address of y is 0x7ff7b207d6d8

The value of y is 6
The address of y is 0x7ff7b207d6d8
```

Explanation:

```
[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % gcc -no-pie -o static-nopie static.c
clang: warning: argument unused during compilation: '-no-pie' [-Wunused-command-line-argument]
```

The results are essentially the same because Apple Clang on macOS ignores the **-no-pie** option, as shown by the warning that the argument was unused. Therefore, the option does not change the program's behavior on our MacBook.

The static version still produces **6, 7, 8**, while the non-static version produces **6, 6, 6**. The memory address remains the same within each execution but may change between executions because of ASLR.

2. Extern Storage Class

Step 1: Compile and run this program 3 times, copy the output and put it in your report, explain why the result show up that way

Code:

```
UW PICO 5.09                                     File: extern.c

#include <stdio.h>
#include <stdlib.h>

int x = 20;

void display()
{
    extern int x;

    printf(" Display value of x: %d\n", x);
    printf(" Adress of x (in display function): %p\n\n", &x);
}

void main()
{
    extern int x;

    printf(" Print value of x: %d\n", x);
    printf(" Adress of x (in main function): %p\n\n", &x);

    display();
}
```

[^]G Get Help [^]O WriteOut [^]R Read File [^]Y Prev Pg [^]K Cut Text [^]C Cur Pos
[^]X Exit [^]J Justify [^]W Where is [^]V Next Pg [^]U UnCut Tex [^]T To Spell

Explanation:

The variable **x** is declared as a global variable with an initial value of 20. The **extern** keyword inside the **main()** and **display()** functions indicates that the variable **x** is defined elsewhere. Therefore, both functions refer to the same global variable rather than creating separate variables.

As a result, both functions print the value 20 and show the same memory address within each program execution. For example, in the first execution, both `main()` and `display()` show the address `0x10868c000`.

The address changes between different executions because the operating system uses Address Space Layout Randomization (ASLR), which can assign a different virtual memory address to the program each time it runs.

Step 2: Copy the output and put it in your report, explain why the results show up that way, and discuss the difference between both outputs.

Output:

```
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern
Print value of x: 20
Address of x (in main function): 0x10868c000

Display value of x: 20
Address of x (in display function): 0x10868c000

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern
Print value of x: 20
Address of x (in main function): 0x104f91000

Display value of x: 20
Address of x (in display function): 0x104f91000

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern
Print value of x: 20
Address of x (in main function): 0x10ab47000

Display value of x: 20
Address of x (in display function): 0x10ab47000
```

Step 3: Redo step 1-3, but this time compile the program using:

“`gcc -no-pie -o <your executable object name> <your c code name>`”

Are they different? Why or why not

Command used :

```
gcc -no-pie -o extern-nopie extern.c
```

Output:

```
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern-nopie
Print value of x: 20
Address of x (in main function): 0x103cf0000

Display value of x: 20
Address of x (in display function): 0x103cf0000

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern-nopie
Print value of x: 20
Address of x (in main function): 0x108720000

Display value of x: 20
Address of x (in display function): 0x108720000

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./extern-nopie
Print value of x: 20
Address of x (in main function): 0x109a6b000

Display value of x: 20
Address of x (in display function): 0x109a6b000
```

Explanation:

```
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % gcc -no-pie -o extern-nopie extern.c
clang: warning: argument unused during compilation: '-no-pie' [-Wunused-command-line-argument]
```

The output is essentially the same as the normal compilation. Both **main()** and **display()** access the same global variable **X**, so the value is **20** and the memory address is the same within each execution.

The address changes between executions because of ASLR. On our MacBook, Apple Clang ignores the **-no-pie** option, as shown by the compiler warning. Therefore, using **-no-pie** does not produce a significant difference in the output.

3. Understand the memory allocation function by following those steps.

Step 1: Run this program (without '-no-pie'):

```
UW PICO 5.09           File: memory.c           Modified

#include <stdio.h>
#include <stdlib.h>

void main()
{
    int *a;
    float *b;
    int c[10];

    printf("\nAddress of Pointer\n");
    printf(">>>p\n", &a);
    printf(">>>p\n", &b);

    printf("\nEffective Address\n");
    printf(">>>p\n", a);
    printf(">>>p\n", b);

    a = (int*) malloc(10*sizeof(int));

    printf("\nAfter malloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);

    printf("\nArray c\n");
    printf(">>>p\n", c);
    printf(">>>p\n", &c[0]);
    printf(">>>p\n", &c[9]);

    /*
    b = (float*) malloc(10*sizeof(float));

    printf("\nAfter malloc Pointer b\n");
    printf(">>>p\n", b);
    printf(">>>p\n", &b[0]);
    printf(">>>p\n", &b[9]);
    */

    a = (int*) realloc(a, 1000*sizeof(int));

    printf("\nAfter realloc Pointer a\n");
    printf(">>>p\n", a);
    printf(">>>p\n", &a[0]);
    printf(">>>p\n", &a[9]);
    printf(">>>p\n", &a[999]);

    free(a);
}

^G Get Help ^O WriteOut ^R Read File ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit ^J Justify ^W Where is ^V Next Pg ^U UnCut Te ^T To Spell
```

Step 2: Copy the output and put it in your report

```

chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./memory

Address of Pointer
>>0x7ff7bb3b56b8
>>0x7ff7bb3b56b0

Effective Address
>>0x0
>>0x0

After malloc Pointer a
>>0x6000033842a0
>>0x6000033842a0
>>0x6000033842c4

Array c
>>0x7ff7bb3b56c0
>>0x7ff7bb3b56c0
>>0x7ff7bb3b56e4

After realloc Pointer a
>>0x7fda03809800
>>0x7fda03809800
>>0x7fda03809824
>>0x7fda0380a79c
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 %

```

step 3: Explain why the results show up that way?

Explanation:

&a and **&b** show the addresses of the pointer variables themselves. Since the system is 64-bit, each pointer occupies 8 bytes.

Initially, **a** and **b** have not been assigned allocated memory. In our execution, their values were **0x0**, but uninitialized pointer values are not guaranteed.

After **malloc(10*sizeof(int))**, memory for 10 integers is allocated. Since **sizeof(int)** is 4 bytes, the allocation is 40 bytes. **a** and **&a[0]** have the same address because **a** points to the first element. **&a[9]** is 36 bytes after **&a[0]**.

Similarly, **c** and **&c[0]** have the same address because **c** is an array. However, **c** is a fixed local array, while **a** uses dynamically allocated memory.

After **realloc()**, **a** is resized from 10 integers (40 bytes) to 1000 integers (4000 bytes). Its address changed in our execution because the memory was moved to a new location.

step 4: Modify the code by uncomment 2 places, and re-run this program

Modified Code:

```

#include <stdio.h>
#include <stdlib.h>

void main()
{
    int *a;
    float *b;
    int c[10];

    printf("\nAddress of Pointer\n");
    printf(">>%p\n", &a);
    printf(">>%p\n", &b);

    printf("\nEffective Address\n");
    printf(">>%p\n", a);
    printf(">>%p\n", b);

    a = (int*) malloc(10*sizeof(int));

    printf("\nAfter malloc Pointer a\n");
    printf(">>%p\n", a);
    printf(">>%p\n", &a[0]);
    printf(">>%p\n", &a[9]);

    printf("\nArray c\n");
    printf(">>%p\n", c);
    printf(">>%p\n", &c[0]);
    printf(">>%p\n", &c[9]);

    b = (float*) malloc(10*sizeof(float));

    printf("\nAfter malloc Pointer b\n");
    printf(">>%p\n", b);
    printf(">>%p\n", &b[0]);
    printf(">>%p\n", &b[9]);

    a = (int*) realloc(a, 1000*sizeof(int));

    printf("\nAfter realloc Pointer a\n");
    printf(">>%p\n", a);
    printf(">>%p\n", &a[0]);
    printf(">>%p\n", &a[9]);
    printf(">>%p\n", &a[999]);

    free(a);
    free(b);
}

```

```

^G Get Help ^O WriteOut ^R Read Fil ^Y Prev Pg ^K Cut Text ^C Cur Pos
^X Exit      ^J Justify  ^W Where is ^V Next Pg ^U UnCut Te ^T To Spell

```

step 5: Copy the output and put it in your report

Output:

```
[chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 % ./memory

Address of Pointer
>>0x7ff7b45a66b8
>>0x7ff7b45a66b0

Effective Address
>>0x0
>>0x0

After malloc Pointer a
>>0x600002dac000
>>0x600002dac000
>>0x600002dac024

Array c
>>0x7ff7b45a66c0
>>0x7ff7b45a66c0
>>0x7ff7b45a66e4

After malloc Pointer b
>>0x600002dac030
>>0x600002dac030
>>0x600002dac054

After realloc Pointer a
>>0x7feea8009800
>>0x7feea8009800
>>0x7feea8009824
>>0x7feea800a79c
chichachayanit@MacBook-Air-khxng-Chicha CPE334_Lab4 %
```

step 6: Explain why the result show up that way? And why is the result different from step 2.

Explanation:

After uncommenting the code for **b**, the program allocates memory for 10 **float** values. On our system, both **int** and **float** occupy 4 bytes, so the allocation for **b** is 40 bytes.

b and **&b[0]** have the same address because **b** points to the first element. The address of **&b[9]** is 36 bytes after **&b[0]**.

The result differs from Step 2 because Step 4 additionally allocates memory for **b**, creating another dynamically allocated memory block. The exact addresses can differ because they are determined by the memory allocator.

The **realloc()** operation also changes the address of **a** in our execution because the allocation increases from 40 bytes to 4000 bytes, so the memory may need to be moved to another location.